Отчет по лабораторной работе №3

“Табулированные функции: массивы и списки"

Выполнила: Качкуркина Арина Валерьевна

Группа: 6204-010302D

Год: 2025

Задание 1

Я ознакомилась со следующими классами исключений, входящих в API Java:

• java.lang.Exception

• java.lang.IndexOutOfBoundsException

• java.lang.ArrayIndexOutOfBoundsException

• java.lang.IllegalArgumentException

• java.lang.IllegalStateException

Задание 2

В пакете functions я реализовала два класса исключений, которые обрабатывают возможные проблемы при работе с табулированными функциями:

- **FunctionPointIndexOutOfBoundsException**- это исключение для случаев, когда программа пытается обратиться к несуществующей точке по индексу(наследование от IndexOutOfBoundsException).

- **InappropriateFunctionPointException**- это для ситуаций, когда нарушается упорядоченность точек или добавляется точка с уже существующей координатой X. Так как это логическая ошибка, а не системная (унаследованно от Exception).

При создании этих классов я использовала возможности среды разработки: автоматическое наследование, генерацию конструкторов и проверку корректности кода.

Задание 3

В класс ArrayTabulatedFunction я добавила проверки и исключения, чтобы сделать его более надежным. В каких ситуациях какие исключения должны выбрасываться:

-конструкторы теперь проверяют входные параметры и не позволяют создать функцию с некорректными границами или недостаточным количеством точек.

-методы работы с точками защищены от неправильных индексов - если передать индекс за пределами массива, будет выброшено понятное исключение.

-методы изменения точек следят за сохранением упорядоченности по X. Я добавила проверки, чтобы новая точка не нарушала последовательность координат.

-удаление точек ограничено - нельзя удалить точку, если после этого останется меньше двух точек, так как функция должна иметь как минимум две точки для определения.

Задание 4

Реализация связного списка. Я создала внутренний класс FunctionNode, который содержит данные точки и ссылки на соседние узлы. Оптимизация доступа к элементам - добавила поле lastNode для запоминания последнего используемого узла, что ускоряет последовательный доступ.

Основные методы работы со списком:
- *getNodeByIndex()* - находит узел по индексу, используя оптимизацию через *lastNode*
- *addNodeToTail*() - добавляет узел в конец списка
- *addNodeByIndex*() - вставляет узел в нужную позицию
- *deleteNodeByIndex*() - удаляет узел, корректно обновляя ссылки

Задание 5

Для класса LinkedListTabulatedFunction я реализовала все те же методы, что и для ArrayTabulatedFunction, но с использованием связного списка. Это позволило сравнить две разные реализации одной и той же

функциональности. Особенно интересно было адаптировать алгоритмы для работы со списком вместо массива.

Задание 6

Я создала общий интерфейс TabulatedFunction. Это улучшило архитектуру программы - теперь можно работать с разными реализациями табулированных функций через единый интерфейс. ArrayTabulatedFunction и LinkedListTabulatedFunction стали реализациями этого интерфейса.

Задание 7

Тестирование показало, что обе реализации работают корректно и выдают одинаковые результаты. Я проверила различные сценарии:

- Корректные операции с точками
- Граничные случаи
- Обработку исключений

Особенно важно было убедиться, что линейная интерполяция работает правильно в обеих реализациях.

Работа успешно завершена, все требования задания выполнены.

РЕЗУЛЬТАТ РАБОТЫ МОЕЙ ПРОГРАММЫ:

Тестирование ArrayTabulatedFunction

Исходная функция

Левая граница: 0.0

Правая граница: 4.0

Кол-во точек: 5

Точки функции: [(0.0, 0.0), (1.0, 1.0), (2.0, 2.0), (3.0, 3.0), (4.0, 4.0)]

Проверка значений функции

f(-1.0) = NaN

f(0.5) = 0.5

f(1.7) = 1.7

f(2.2) = 2.2

f(3.4) = 3.4

f(5.5) = NaN

Проверка операций с точками

1. Добавляем точку (2.2, 2.2):

Точки функции: [(0.0, 0.0), (1.0, 1.0), (2.0, 2.0), (2.2, 2.2), (3.0, 3.0), (4.0, 4.0)]

f(2.2) = 2.2

2. Удаляем точку с индексом 1:

Точки функции: [(0.0, 0.0), (2.0, 2.0), (2.2, 2.2), (3.0, 3.0), (4.0, 4.0)]

3. Изменяем у точки с индексом 2 на 2.5:

Точки функции: [(0.0, 0.0), (2.0, 2.0), (2.2, 2.5), (3.0, 3.0), (4.0, 4.0)]

Итоговый результат

f(2.2) = 2.5

Точки функции: [(0.0, 0.0), (2.0, 2.0), (2.2, 2.5), (3.0, 3.0), (4.0, 4.0)]

Тестирование LinkedListTabulatedFunction

Исходная функция

Левая граница: 0.0

Правая граница: 4.0

Кол-во точек: 5

Точки функции: [(0.0, 0.0), (1.0, 1.0), (2.0, 2.0), (3.0, 3.0), (4.0, 4.0)]

Проверка значений функции

f(-1.0) = NaN

f(0.5) = 0.5

f(1.7) = 1.7

f(2.2) = 2.2

f(3.4) = 3.4

f(5.5) = NaN

Проверка операций с точками

1. Добавляем точку (2.2, 2.2):

Точки функции: [(0.0, 0.0), (1.0, 1.0), (2.0, 2.0), (2.2, 2.2), (3.0, 3.0), (4.0, 4.0)]

f(2.2) = 2.2

2. Удаляем точку с индексом 1:

Точки функции: [(0.0, 0.0), (2.0, 2.0), (2.2, 2.2), (3.0, 3.0), (4.0, 4.0)]

3. Изменяем у точки с индексом 2 на 2.5:

Точки функции: [(0.0, 0.0), (2.0, 2.0), (2.2, 2.5), (3.0, 3.0), (4.0, 4.0)]

Итоговый результат

f(2.2) = 2.5

Точки функции: [(0.0, 0.0), (2.0, 2.0), (2.2, 2.5), (3.0, 3.0), (4.0, 4.0)]

Проверка исключений

1. Неправильные границы функции (левая >= правой):

Получено: Левая граница должна быть меньше правой

2. Слишком мало точек (меньше 2):

Получено: Количество точек должно быть не менее 2

3. Несуществующий индекс точки:

Получено: Индекс 5 вне границ [0, 2]

4. Нарушение порядка точек по X:

Получено: Новый X должен быть меньше следующего

5. Добавление точки с уже существующим X:

Получено: Точка с X=1.0 уже есть

6. Удаление точки когда их останется слишком мало:

Получено: Нельзя удалить точку: должно остаться минимум 2 точки